

DBMS PRACTICAL 4

```
1 • DROP DATABASE IF EXISTS bookflow_db;
2 • CREATE DATABASE bookflow_db;
3 • USE bookflow_db;
4 • CREATE TABLE Books (
5     book_id INT AUTO_INCREMENT PRIMARY KEY,
6     title VARCHAR(200) NOT NULL,
7     author VARCHAR(150) NOT NULL,
8     genre VARCHAR(100),
9     description TEXT,
10    published_year INT
11 );
12 • INSERT INTO Books (title, author, genre, description, published_year) VALUES
13 ('The Martian', 'Andy Weir', 'Science Fiction', 'An astronaut struggles to survive alone on Mars.', 2011),
14 ('Dune', 'Frank Herbert', 'Science Fiction', 'A story involving planets, politics, survival and space.', 1965),
15 ('The Hitchhikers Guide to the Galaxy', 'Douglas Adams', 'Science Fiction', 'A humorous adventure across space and alien worlds.', 1979),
16 ('Pride and Prejudice', 'Jane Austen', 'Romance', 'A classic story about relationships and social expectations.', 1813),
17 ('The Great Gatsby', 'F. Scott Fitzgerald', 'Classic', 'A story of ambition, wealth, love and the American dream.', 1925),
18 ('Foundation', 'Isaac Asimov', 'Science Fiction', 'A futuristic story about mathematics, civilization and space.', 1951),
19 ('The Hobbit', 'J.R.R. Tolkien', 'Fantasy', 'A fantasy adventure involving Bilbo Baggins and a dangerous journey.', 1937),
20 ('Enders Game', 'Orson Scott Card', 'Science Fiction', 'A young student is trained for an interstellar conflict.', 1985),
21 ('Harry Potter and the Sorcerers Stone', 'J.K. Rowling', 'Fantasy', 'A young wizard begins his magical education.', 1997),
```

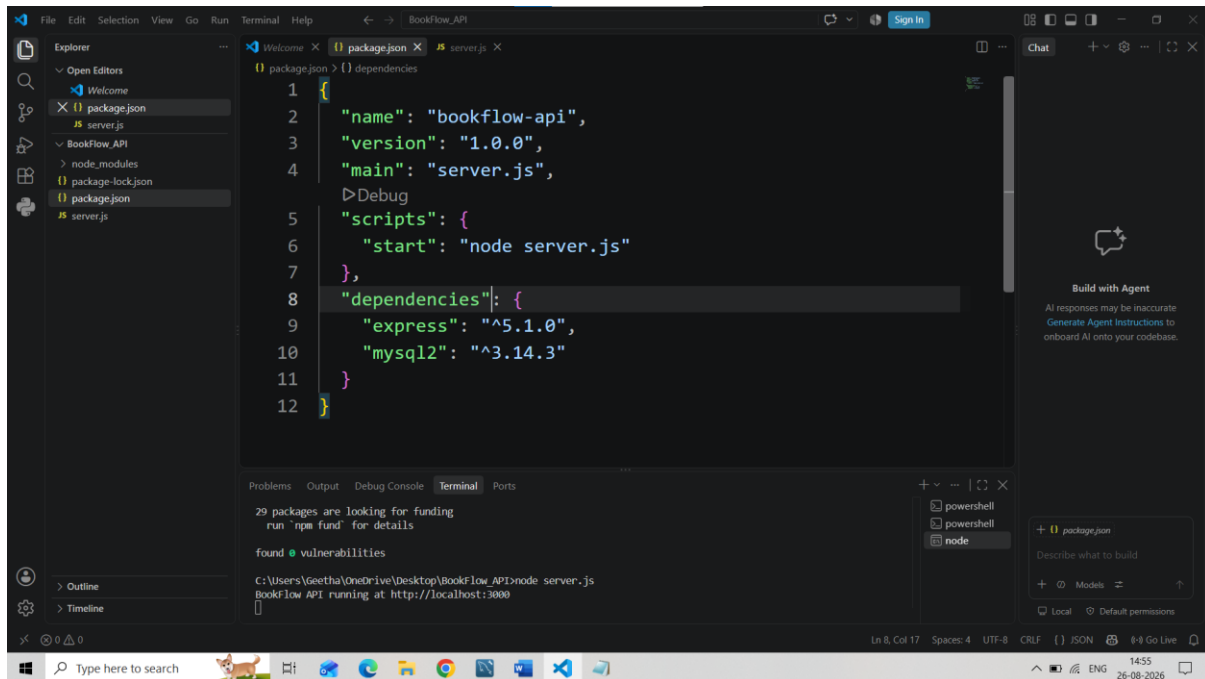
OUTPUT:

✓	2	14:33:00	DROP DATABASE IF EXISTS bookflow_db	4 row(s) affected
✓	3	14:33:05	CREATE DATABASE bookflow_db	1 row(s) affected
✓	4	14:33:08	USE bookflow_db	0 row(s) affected
✓	5	14:33:31	CREATE TABLE Books (book_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(200) NOT N...	0 row(s) affected
✓	6	14:34:11	INSERT INTO Books (title, author, genre, description, published_year) VALUES ('The Martian', 'Andy Weir', 'Sci...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0
✓	7	14:34:35	ALTER TABLE Books ADD COLUMN embedding JSON	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
✓	8	14:34:35	UPDATE Books SET embedding = JSON_ARRAY(0.90, 0.80, 0.20) WHERE book_id = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	9	14:34:35	UPDATE Books SET embedding = JSON_ARRAY(0.85, 0.75, 0.25) WHERE book_id = 2	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	10	14:34:35	UPDATE Books SET embedding = JSON_ARRAY(0.80, 0.70, 0.30) WHERE book_id = 3	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	11	14:34:35	UPDATE Books SET embedding = JSON_ARRAY(0.10, 0.20, 0.90) WHERE book_id = 4	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
✓	12	14:34:36	UPDATE Books SET embedding = JSON_ARRAY(0.15, 0.30, 0.80) WHERE book_id = 5	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0

```
35 • SELECT
36     book_id, title, author, genre,
37     ROUND(
38         (
39             CAST(JSON_EXTRACT(embedding, '$[0]') AS DECIMAL(10,6)) * 0.90 +
40             CAST(JSON_EXTRACT(embedding, '$[1]') AS DECIMAL(10,6)) * 0.80 +
41             CAST(JSON_EXTRACT(embedding, '$[2]') AS DECIMAL(10,6)) * 0.20
42         ) / (
43             SQRT(
44                 POW(CAST(JSON_EXTRACT(embedding, '$[0]') AS DECIMAL(10,6)), 2) +
45                 POW(CAST(JSON_EXTRACT(embedding, '$[1]') AS DECIMAL(10,6)), 2) +
46                 POW(CAST(JSON_EXTRACT(embedding, '$[2]') AS DECIMAL(10,6)), 2)
47             ) * SQRT(POW(0.90, 2) + POW(0.80, 2) + POW(0.20, 2))
48         ), 4
49     ) AS cosine_similarity
50 FROM Books
51 WHERE embedding IS NOT NULL
52 ORDER BY cosine_similarity DESC
53 LIMIT 3;
```

OUTPUT:

	book_id	title	author	genre	cosine_similarity
▶	1	The Martian	Andy Weir	Science Fiction	1
	2	Dune	Frank Herbert	Science Fiction	0.9986
	8	Enders Game	Orson Scott Card	Science Fiction	0.9983



```
1 const express = require("express");
2 const mysql = require("mysql2/promise");
3 const app = express();
4 const PORT = 3000;
5
6 app.use(express.json());
7
8 const pool = mysql.createPool({
9   host: "localhost",
10  user: "root",
11  password: "YOUR_MYSQL_PASSWORD", // <--- UPDATE THIS
12  database: "bookflow_db",
13  waitForConnections: true,
14  connectionLimit: 10,
15  queueLimit: 0
```

```

18 app.get("/", (req, res) => {
19     res.json({
20         message: "BookFlow Intelligent Catalog API",
21         status: "Running",
22         endpoints: {
23             search: "GET /v1/books/search",
24             createBook: "POST /v1/books"
25         }
26     });
27 });
28

```

```

29 app.get("/v1/books/search", async (req, res) => {
30     try {
31         const q = req.query.q;
32         const limit = parseInt(req.query.limit || "10");
33
34         if (!q || q.trim() === "") {
35             return res.status(400).json({
36                 error: { code: "INVALID_QUERY", message: "Q"
37             });
38         }
39
40         if (isNaN(limit) || limit < 1 || limit > 50) {
41             return res.status(400).json({
42                 error: { code: "INVALID_LIMIT", message: "1
43             });
44         }
45     } catch (e) {
46         return res.status(500).json({
47             error: { code: "INTERNAL_SERVER_ERROR", message: "Internal Server Error"
48         });
49     }
50 });
51

```

```

46 : x = 0.90, y = 0.80, z = 0.20;
47
48 : sql = `
49 :T
50 book_id, title, author, genre, description, published_year,
51 ROUND(
52     (
53         CAST(JSON_EXTRACT(embedding, '$[0]') AS DECIMAL(10,6)) *
54         CAST(JSON_EXTRACT(embedding, '$[1]') AS DECIMAL(10,6)) *
55         CAST(JSON_EXTRACT(embedding, '$[2]') AS DECIMAL(10,6)) *
56     ) / (
57         SQRT(
58             POW(CAST(JSON_EXTRACT(embedding, '$[0]') AS DECIMAL(10,6))
59

```

```

57         Sqrt(
58             POW(CAST(JSON_EXTRACT(embedding, '$[0]') AS DECIMAL(10, 4)), 2) +
59             POW(CAST(JSON_EXTRACT(embedding, '$[1]') AS DECIMAL(10, 4)), 2) +
60             POW(CAST(JSON_EXTRACT(embedding, '$[2]') AS DECIMAL(10, 4)), 2)
61         ) * Sqrt(POW(?, 2) + POW(?, 2) + POW(?, 2))
62     ), 4
63 ) AS similarity
64 Books
65 WHERE embedding IS NOT NULL
66 ORDER BY similarity DESC
67 LIMIT 10;
68

```

```

70     res.status(200).json({ query: q, count: rows.length,
71     } catch (error) {
72         console.error(error);
73         res.status(500).json({
74             error: { code: "INTERNAL_SERVER_ERROR", message:
75             });
76     }
77 });
78
79 app.post("/v1/books", async (req, res) => {
80     try {
81         const { title, author, genre, description, published
82
83         if (!title || !author || !genre) {

```

```

84             return res.status(400).json({
85                 error: { code: "INVALID_BOOK", message: "tit
86             });
87         }
88
89         const sql = `INSERT INTO Books (title, author, genre,
90         const [result] = await pool.execute(sql, [
91             title, author, genre, description || null, publis
92             embedding ? JSON.stringify(embedding) : null
93         ]);
94
95         res.status(201).json({
96             message: "Book created successfully",

```

Output:

```
29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Geetha\OneDrive\Desktop\BookFlow_API>node server.js
BookFlow API running at http://localhost:3000
█
```

Output:

```
Pretty-print ☐
{"message":"BookFlow Intelligent Catalog API","status":"Running","endpoints":{"search":"GET /v1/books/search","createBook":"POST /v1/books"}}
```

Swagger:

```
1  openapi: 3.0.3
2
3  info:
4    title: BookFlow Intelligent Catalog API
5    version: 1.0.0
6    description: REST API for semantic book search using mock vectors stored
7      in MySQL.
8
9  servers:
10    - url: http://localhost:3000
11
12  tags:
13    - name: Books
14      description: Book operations
15
16  paths:
17    /v1/books/search:
18      get:
19        tags:
20          - Books
21        summary: Search books using semantic similarity
22        parameters:
23          - name: q
24            in: query
25            required: true
26            description: Natural language search query
27            schema:
28              type: string
29              minLength: 1
30              example: Stories about space
31          - name: limit
```

```
34         required: false
35         description: Maximum number of results
36         schema:
37             type: integer
38             minimum: 1
39             maximum: 50
40             default: 10
41             example: 3
42
43     responses:
44         '200':
45             description: Successful search
46             content:
47                 application/json:
48                     schema:
49                         $ref: '#/components/schemas/SearchResponse'
50
51         '400':
52             description: Bad Request
53             content:
54                 application/json:
55                     schema:
56                         $ref: '#/components/schemas/ErrorResponse'
57
58     /v1/books:
59         post:
60             tags:
61                 - Books
62             summary: Create a new book
63
64             requestBody:
65                 required: true
```

```

66         content:
67             application/json:
68                 schema:
69                     $ref: '#/components/schemas/CreateBookRequest'
70
71         responses:
72             '201':
73                 description: Book created successfully
74             '400':
75                 description: Bad Request
76                 content:
77                     application/json:
78                         schema:
79                             $ref: '#/components/schemas/ErrorResponse'
80
81 components:
82     schemas:
83         Book:
84             type: object
85             properties:
86                 book_id:
87                     type: integer
88                 title:
89                     type: string
90                 author:
91                     type: string
92                 genre:
93                     type: string
94                 description:
95                     type: string
96                 published_year:

```

```

104     searchresponse:
105         type: object
106         properties:
107             query:
108                 type: string
109             count:
110                 type: integer
111             results:
112                 type: array
113             items:
114                 $ref: '#/components/schemas/Book'
115
116     CreateBookRequest:
117         type: object
118         required:
119             - title
120             - author
121             - genre
122         properties:
123             title:
124                 type: string
125             author:
126                 type: string
127             genre:
128                 type: string
129             description:
130                 type: string
131             published_year:
132                 type: integer
133             embedding:
134                 type: array
135                 minItems: 3
136                 maxItems: 3

```

```

137             items:
138                 type: number
139
140     ErrorResponse:
141         type: object
142         properties:
143             error:
144                 type: object
145                 properties:
146                     code:
147                         type: string
148                     message:
149                         type: string

```



GET

/v1/books/search

Search books using semantic similarity

Parameters

Cancel

Name	Description
q * required	Natural language search query
string (query)	Stories about space
limit	Maximum number of results
integer (query)	3

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://localhost:3000/v1/books/search?q=Stories%20about%20space&limit=3' \
-H 'accept: application/json'
```

```
'http://localhost:3000/v1/books/search?q=Stories%20about%20space&limit=3' \
-H 'accept: application/json'
```



Request URL

```
http://localhost:3000/v1/books/search?q=Stories%20about%20space&limit=3
```

Server response

Code

Details

200

Response body

```
{
  "query": "Stories about space",
  "count": 3,
  "results": [
    {
      "book_id": 1,
      "title": "The Martian",
      "author": "Andy Weir",
      "genre": "Science Fiction",
      "description": "An astronaut struggles to survive alone on Mars.",
      "published_year": 2011,
      "similarity": 1
    },
    {
      "book_id": 2,
      "title": "Dune",
      "author": "Frank Herbert",
      "genre": "Science Fiction",
      "description": "A story involving planets, politics, survival and space.",
      "published_year": 1965,
      "similarity": 0.9986
    },
    {
      "book_id": 8,
      "title": "Enders Game",
      "author": "Orson Scott Card",
      "genre": "Science Fiction",
      "description": "A story about a young boy who becomes a military leader."
    }
  ]
}
```



Download

Code	Description	Link
200	Successful search	No /
Media type application/json Controls Accept header.		
Example Value Schema		
<pre>{ "query": "string", "count": 0, "results": [{ "book_id": 0, "title": "string", "author": "string", "genre": "string", "description": "string", "published_year": 0, "similarity": 0 }] }</pre>		
400	Bad Request	No /
Media type		

400	Bad Request	
Media type		
application/json		
Example Value Schema		
<pre>{ "error": { "code": "string", "message": "string" } }</pre>		